

TECHNICAL DEVELOPMENT HANDOFF

WebAuthn-Secured Attendance for VE WorkLog

Device registration, passkey verification, anti-buddy-punching controls, limitations, and implementation specification

Document field	Value
Application	VE WorkLog - Vertex Electronics and Vision Engineering
Production domain	https://tasks.vertex.pk
Prepared for	AI/developer implementation handoff
Prepared date	10 August 2026
Status	Proposed WebAuthn enhancement; employee Attendance navigation currently paused

Executive decision

Add WebAuthn user verification and an admin-approved attendance credential to the existing GPS attendance flow. This materially reduces password-only buddy punching, but it cannot guarantee that fingerprint or Face ID - rather than device PIN - was used. For deliberate collusion involving the approved phone and its PIN, add live selfie/liveness or dedicated biometric hardware.

1. Purpose and scope

This document consolidates the complete discussion about adding WebAuthn/passkey-based identity verification to the existing VE WorkLog attendance module. It is written so another AI or developer can continue implementation without the original conversation.

- Preserve the existing username/password login and JWT session model.
- Add a second, attendance-specific authorization step using an approved WebAuthn credential.
- Require the approved credential for both normal check-in and normal checkout.
- Retain GPS validation and existing manual/admin approval fallbacks.
- Allow one active approved attendance credential per employee by policy.
- Do not store fingerprints, facial templates, device PINs, or other biometric data.

2. Existing VE WorkLog attendance context

The WebAuthn feature extends an already implemented browser-based attendance module. The current attendance rules and workflows must remain operational unless explicitly changed during development.

Current control	Existing behavior
Office location	31.441300523433583, 74.32441912480384; shared VE/VSN office
Office radius	Approximately 100 metres, verified server-side with browser GPS coordinates
Check-in window	Before 7:00 AM PKT is blocked
Present	Check-in from 7:00 AM through 9:35 AM PKT
Late	Check-in after 9:35 AM and before 11:00 AM PKT
Half Day	Check-in at or after 11:00 AM PKT
Monthly deductions	3 lates = 1 absence; 2 half-days = 1 absence
Normal checkout	Employee must normally be at the office location
Early Leave / Remote	Checkout outside the office requires mandatory remarks and admin approval
Manual check-in	Used when location is denied/unavailable; mandatory reason and admin approval
Pending check-in	Employee may still submit checkout while check-in approval is pending
Administration	Daily/monthly reporting, approvals, manual correction, PDF export, user attendance toggle

Current testing state

The Attendance item is intentionally hidden from employee navigation while testing is paused. Admin Attendance screens remain available. When WebAuthn testing resumes, restore attendance in both employee navigation definitions: the route allow-list (empViews) and the visible sidebar list (empltems) in public/app.js.

3. Threat model: buddy punching

GPS validates where a device is located; it does not prove who is operating the device. Under the current system, a colleague at the office can mark attendance for an absent employee if that colleague has the employee account credentials.

3.1 What WebAuthn prevents

- A colleague who knows the employee portal password but uses an unregistered phone cannot complete attendance because the approved private credential is missing.
- A copied password is insufficient because every attendance action requires a fresh server challenge signed by the approved WebAuthn credential.
- A credential registered for another website cannot be replayed against tasks.vertex.pk because the signature is bound to the relying-party domain and expected origin.

3.2 What WebAuthn does not fully prevent

- If the employee hands over the approved phone and also shares the device PIN, the colleague may satisfy user verification.
- The website cannot reliably force fingerprint-only or Face-ID-only verification. It asks for user verification; the device chooses fingerprint, face, PIN, pattern, computer password, or security-key PIN.
- The server receives a verified/not-verified result, not the biometric modality.
- Some passkeys can synchronize through Apple iCloud Keychain, Google Password Manager, or another credential provider; one WebAuthn credential is not always equivalent to one physical device.
- An unlocked or deliberately shared device and deliberate cooperation remain risks in any browser-only design.

4. Recommended layered security model

Layer	Purpose	Decision
JWT login	Identifies the portal account	Keep existing login
Approved WebAuthn credential	Proves control of the registered attendance credential	Required for normal check-in/out
User verification	Requires device-level fingerprint/face/PIN/password	Set userVerification to required
Office GPS	Confirms physical proximity to office	Keep existing server-side radius check
GPS accuracy	Rejects unusably imprecise locations	Add an explicit maximum accuracy threshold
Admin approval	Prevents employees from silently registering replacement credentials	Required for registration/replacement

Layer	Purpose	Decision
Audit metadata	Supports investigation	Record credential, time, coordinates, accuracy, IP, user agent
Optional live selfie/liveness	Deters collusion with approved phone/PIN	Phase 2 or risk-triggered
Dedicated biometric terminal	Strongest fingerprint/face enforcement	Future option if strict identity proof is required

5. User experience and application states

5.1 Employee device states

State	Employee interface	Attendance behavior
No device	Register this device	Normal attendance blocked until approved
Pending	Device details and Awaiting admin approval	Normal attendance blocked; manual request available
Approved	Approved status and attendance controls	WebAuthn + GPS required
Rejected	Rejection status and new request action	Normal attendance blocked
Revoked	Register replacement device	Old credential cannot be used
Replacement pending	Old/new status and request reason	Apply management policy on whether old device remains active

5.2 Registration flow

1. Employee signs in on the intended attendance phone and opens Attendance.
2. Employee selects Register this device.
3. Server creates a short-lived, single-use WebAuthn registration challenge.
4. Browser invokes the platform authenticator; the phone requests fingerprint, Face ID, screen-lock PIN, or another supported verification method.
5. The private key remains in the authenticator; the application receives a public credential response.
6. Server verifies the challenge, expected origin, relying-party ID, and user verification result.
7. Credential is stored with status Pending and appears in Admin Device Approvals.
8. Admin verifies the employee and approves or rejects the credential.

5.3 Normal check-in/check-out flow

1. Employee selects Check In or Check Out.
2. Frontend requests authentication options from the server.
3. Server issues a new short-lived, single-use challenge restricted to the employee approved credential.
4. Browser requests device verification and returns a signed assertion.
5. Frontend obtains current GPS latitude, longitude, and accuracy.
6. Frontend submits the signed WebAuthn assertion and GPS payload together.
7. Server verifies WebAuthn first, then GPS/radius/accuracy, then applies attendance time rules.
8. Server updates the credential signature counter and records the attendance event and audit data.

5.4 Buddy-punching outcome

Expected outcome

A colleague can log into the absent employee account and may be physically inside the office radius, but attendance is blocked because the approved private credential is unavailable. The employee must use the approved device or submit a manual request for admin review.

6. WebAuthn technical configuration

Setting	Production value / recommendation
Relying-party name	VE WorkLog
Relying-party ID	tasks.vertex.pk
Expected origin	https://tasks.vertex.pk
Registration attestation	none initially; avoids unnecessary device-identification complexity
Authenticator attachment	platform preferred
Resident key	preferred
User verification	required for registration and authentication
Challenge lifetime	Recommended 3-5 minutes; one-time use
Credential policy	One active approved credential per employee
Transport storage	Store authenticator transports when provided
Recommended Node package	@simplewebauthn/server; browser helper optional
HTTPS	Mandatory; already available at tasks.vertex.pk

6.1 Registration options example

```
const options = await generateRegistrationOptions({
  rpName: 'VE WorkLog',
  rpID: 'tasks.vertex.pk',
  userName: user.email,
  userDisplayName: user.name,
  attestationType: 'none',
  authenticatorSelection: {
    authenticatorAttachment: 'platform',
    residentKey: 'preferred',
    userVerification: 'required'
  }
});
```

6.2 Authentication verification example

```
const verification = await verifyAuthenticationResponse({
  response: webauthnResponse,
  expectedChallenge: savedChallenge,
  expectedOrigin: 'https://tasks.vertex.pk',
  expectedRPID: 'tasks.vertex.pk',
  credential: storedCredential,
  requireUserVerification: true
});
```

The attendance request should carry latitude, longitude, browser-reported accuracy, and the signed WebAuthn assertion. Bind the saved challenge to the intended action (check-in or checkout) and employee account.

7. Proposed database design

7.1 attendance_credentials

Column	Purpose
id	Primary key
user_id	Employee owner; foreign key to users
credential_id	Unique WebAuthn credential identifier
public_key	Credential public key in binary form
counter	Signature counter used during authentication verification

Column	Purpose
transports	Optional browser-reported authenticator transports
device_name	Admin/employee-friendly label, not a security assertion
browser_info	User-agent/device description for audit only
status	pending, approved, rejected, revoked
registered_at	Registration timestamp
approved_by / approved_at	Admin approval audit
revoked_at / revocation_reason	Revocation audit

7.2 webauthn_challenges

Store registration and authentication challenges server-side. Each record should contain user_id, challenge, purpose, expiry, and optionally the attendance action being authorized. Challenges must expire quickly and be deleted or marked consumed after one successful use.

7.3 Attendance audit additions

- Credential ID or credential record ID used for the event.
- Boolean/user-verification result and verification timestamp.
- GPS accuracy supplied by the browser and distance calculated server-side.
- Request IP address and user agent for anomaly review, with an appropriate retention policy.
- Verification mode: webauthn, manual, or admin override.

8. Proposed API surface

Endpoint	Purpose	Access
POST /api/webauthn/register/options	Generate registration challenge/options	Logged-in employee
POST /api/webauthn/register/verify	Verify response and save pending credential	Logged-in employee
POST /api/webauthn/auth/options	Generate check-in/out authentication challenge	Logged-in employee
POST /api/webauthn/auth/verify	Optional separate assertion verification endpoint	Logged-in employee

Endpoint	Purpose	Access
GET /api/webauthn/device	Return employee credential/approval status	Logged-in employee
POST /api/webauthn/device/replace	Request replacement with mandatory reason	Logged-in employee
GET /api/webauthn/admin/devices	List pending/approved/revoked credentials	Admin
PUT /api/webauthn/admin/devices/:id/approve	Approve credential; enforce one-active policy	Admin
PUT /api/webauthn/admin/devices/:id/reject	Reject pending credential	Admin
PUT /api/webauthn/admin/devices/:id/revoke	Revoke approved credential	Admin
POST /api/attendance/checkin	Verify assertion + GPS and record check-in	Employee
POST /api/attendance/checkout	Verify assertion + GPS and record checkout	Employee

Security requirement

The server must generate and verify every WebAuthn challenge. A frontend-only biometric success message is not secure and must never be accepted as proof.

9. Server-side validation sequence

1. Authenticate the JWT and load the active employee.
2. Confirm attendance is enabled for the employee and an approved credential exists.
3. Load a matching, unexpired, unused challenge with the correct purpose/action.
4. Verify credential ID, signed challenge, expected origin, RP ID, and user verification.
5. Check the credential is still approved and not revoked.
6. Validate and update the signature counter using the WebAuthn library result.
7. Validate numeric latitude, longitude, accuracy, and office distance server-side.
8. Apply check-in window, late, half-day, manual, and checkout rules.
9. Record attendance and immutable audit metadata in one database transaction where practical.
10. Consume the challenge so it cannot be replayed.

10. Device replacement and recovery

- Employee submits Replace Device with a mandatory reason.
- Admin reviews the request and revokes the old credential before or during approval of the replacement.
- A lost-device report must allow immediate admin revocation.
- The employee registers the new device and waits for approval.
- Manual attendance remains available as an approval-only fallback; it must be visibly marked unverified by WebAuthn.
- Password changes and suspicious-device events should revoke active sessions; credential revocation is a separate action.

11. Privacy and biometric limitations

WebAuthn does not transmit the employee fingerprint, face template, or device PIN to VE WorkLog. The device uses its local security mechanism to authorize use of the private credential. The server receives a cryptographic assertion and user-verification result.

Important limitation

The web application cannot reliably require fingerprint only or Face ID only. With userVerification set to required, the authenticator may use fingerprint, facial recognition, device PIN, screen-lock pattern, computer password, or security-key PIN. The application cannot reliably identify which method was used.

If the organization must enforce fingerprint or face recognition without PIN fallback, consider dedicated office biometric hardware or a managed native mobile application with device attestation and specialized face-liveness verification.

12. Optional selfie/liveness enhancement

For stronger deterrence against deliberate device/PIN sharing, add a live camera step after WebAuthn. A plain selfie is an audit aid but can be spoofed with a photograph. Random liveness prompts, such as turning the head or blinking, are stronger but require careful privacy, retention, consent, and false-rejection handling.

- Recommended use: risk-triggered, manual requests, newly replaced devices, or random spot checks rather than necessarily twice every day.
- Store images securely with restricted admin access and a defined deletion/retention period.
- Do not claim that a basic browser selfie provides certified face recognition.

13. Application files expected to change

File	Expected change
package.json	Add @simplewebauthn/server and any selected browser helper/build dependency
db/schema.sql	Add credential/challenge tables and attendance audit columns
server/routes/webauthn.js	Registration, authentication, device status, replacement, and admin device routes
server/routes/attendance.js	Require verified assertion for normal check-in/out; retain approved manual fallback
server/index.js	Mount /api/webauthn routes
public/app.js	Employee device states, registration ceremony, attendance assertion ceremony, admin approvals
public/style.css	Device status, approval list, replacement and verification UI
Docker deployment	Rebuild image if package dependencies change; docker cp alone is insufficient for new node_modules

14. Implementation phases and estimate

Phase	Scope	Estimate
1. Foundation	Schema, library, challenge storage, WebAuthn utilities	0.5-1 day
2. Registration	Employee registration and admin approval/revoke UI	1 day
3. Attendance integration	Secure check-in/out with GPS and audit metadata	1 day
4. Recovery and exceptions	Replacement, manual fallback, notifications, anomaly flags	0.5-1 day
5. Cross-device QA	Android, iPhone, Windows; deployment and corrections	1 day

Expected production-ready implementation: approximately 3-5 working days. A basic MVP may be available in about 2 working days, but it should not immediately become the official attendance mechanism without cross-device testing and a controlled pilot.

15. Testing matrix

- Register and approve a platform credential on Android Chrome.
- Register and approve a platform credential on iPhone Safari.
- Register and approve Windows Hello on Windows Chrome/Edge if desktop attendance is supported.
- Attempt attendance from another phone using the correct portal password; expect block.
- Attempt with expired, reused, wrong-purpose, and wrong-user challenges; expect block.
- Attempt with a rejected or revoked credential; expect block.
- Test GPS inside radius, outside radius, permission denied, low accuracy, and malformed coordinates.
- Test before 7:00 AM, present, late, and half-day boundaries in PKT.
- Test checkout while manual check-in approval remains pending.
- Test replacement workflow and ensure old credential can no longer authorize attendance after revocation.
- Confirm employee and admin audit displays clearly distinguish WebAuthn, manual, and admin override events.
- Confirm synchronized-passkey behavior on supported ecosystems and document policy implications.

16. Acceptance criteria

- Normal check-in/out cannot complete without an approved, non-revoked credential and required user verification.
- Server validates challenge, origin, RP ID, credential ownership, status, verification result, and GPS independently of the browser UI.
- Challenges are short-lived, single-use, purpose-bound, and replay resistant.
- Only admins can approve, reject, revoke, or replace an employee approved attendance credential.
- One-active-credential policy is enforced transactionally.
- Manual attendance remains possible but requires reason, is marked unverified, and follows admin approval.
- No biometric template, fingerprint, Face ID data, or device PIN is stored by VE WorkLog.
- Employee Attendance navigation is restored only when testing resumes and is updated in both empViews and empItems.
- All existing attendance calculations and admin reports continue to work.

17. Recommended rollout decision

1. Implement registration and admin approval first without enforcing it on attendance.
2. Pilot with a small group using Android, iPhone, and Windows where relevant.
3. Review synchronized-passkey behavior and decide whether one credential or one managed physical device is the actual policy.
4. Enable WebAuthn enforcement for pilot employees while retaining manual approval fallback.
5. Restore the employee Attendance tab when formal testing resumes.
6. After stable operation, enforce WebAuthn for all attendance-enabled employees.
7. Add selfie/liveness only if residual deliberate-collusion risk justifies the privacy and operational burden.

18. Final implementation guidance for another AI/developer

Do not weaken these controls

Never trust a client-provided biometric-success flag, client-provided office-distance result, or client-generated attendance status. Verify WebAuthn and calculate GPS distance/time classification on the server. Keep secrets and SMTP/JWT configuration outside source control.

- Inspect the current GitHub/project state before editing because the attendance module already contains several post-MVP corrections.
- Preserve the separate checkin_remark and checkout_remark fields and distinct Manual Request versus Early Leave / Remote labels.
- Preserve the ability to checkout while check-in approval is pending.
- Preserve company grouping for VTX, VSN, and ALL in daily/monthly/PDF reports.
- Preserve local/PKT date handling; avoid using UTC toISOString() for browser TODAY date selection.
- Rebuild the Docker app image when adding Node dependencies. The production service is vv_app, PostgreSQL is vv_db, and Nginx is vv_nginx.
- The production environment file is /home/ubuntu/vertex-monitoring-app/.env and must not be committed.
- Deploy through the Ubuntu host repository and Docker Compose/container workflow, then verify logs and all WebAuthn ceremonies on the production HTTPS origin.

19. Reference standards

Primary references for implementation: W3C Web Authentication Level 3 specification and the SimpleWebAuthn server/browser documentation. Confirm the installed package version and follow that version API because credential object property names may differ between library releases.

- W3C WebAuthn: <https://www.w3.org/TR/webauthn-3/>
- SimpleWebAuthn documentation: <https://simplewebauthn.dev/docs/>
- MDN Web Authentication API: https://developer.mozilla.org/en-US/docs/Web/API/Web_Authentication_API